

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	G. Santosh Kumar	Reg. No.:	192524020
Section / Batch:	2025 Batch	Date:	04/09/2026

Code of Conduct

I, G. Santosh / 192524020 (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Santosh Kumar

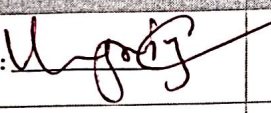
Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

start

↓

Input graph & source

↓

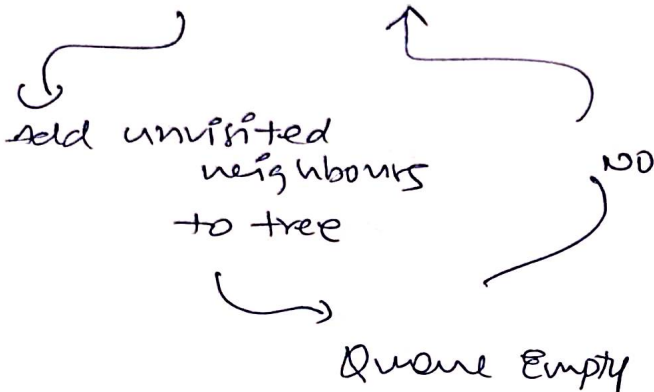
Initialize queue

↓

Visit source

↓

Remove front vertex



```
#include <stdio.h>
```

```
#define MAX 20
```

```
int graph[MAX][MAX];
```

```
int visited[MAX];
```

```
int queue[MAX];
```

```
int front = 0, rear = 0;
```

```
int n;
```

```
void BFS (int start) {
```

```
    visited[start] = 1;
```

```
    queue[rear++] = start;
```

```
    printf ("BFS traversal: ");
```

while (front < rear) {

int current = queue[front++];

printf("BFS traversal");

while (int i = 0; i < n; i++) {

if (graph[current][i] == 1 && visited[i] == 0) {

visited[i] = 1;

queue[rear++] = i;

printf("BFS tree edge: %d → %d", current, i);

}

}

}

printf("\n");

}

int main() {

int start;

printf("Enter number of vertices: ");

scanf("%d", &n);

printf("Enter adjacency matrix: ");

for (int i = 0; i < n; i++) {

for (int j = 0; j < n; j++) {

scanf("%d", &graph[i][j]);

}

}

start with a starting vertex: u ;

Scanf ("%d", &start);

BFS(start);

return;

u

②

start

↳

Read weighted graph
& source

↳

Initialize distance
array

↳
~~set source distance to~~ 0

select unvisited vertex
with min distance

mark vertex visited

All vertices
processed?

no

yes

print
shortest
distances

Relax adjacent
edges

Edges in Minimum Spanning Tree:

$$0-1 : 2$$

$$1-2 : 3$$

$$1-8 : 5$$

$$0-3 : 6$$

$$\text{Total MST cost} = 16$$

Wong